

```
| Welcome to JShell -- Version 9-ea
| For an introduction type: /help intro
```

```
jshell> int x= 10
x ==> 10
```

```
jshell> System.out.println("value of x is: "+ x);
value of x is: 10
```

Native HTTP/2 and Web sockets

Finally, the age old `HttpURLConnection` has been replaced with the new `HttpClient` in Java 9, which provides out-of-the-box support for the HTTP/2 protocol, Web sockets and async HTTP requests.

```
HttpClient client = HttpClient.newHttpClient();

HttpRequest req =
    HttpRequest.newBuilder(URI.create("http://metalop.
com"))
        .header("User-Agent", "Java")
        .GET()
        .build();

HttpResponse<String> response = client.send(req,
HttpResponse.BodyHandler.asString());
System.out.println(response.statusCode());
System.out.println(response.body());
```

Beautiful, isn't it? No `InputStream` or `Reader` is involved — instead, the API offers a `BodyHandler` which allows us to read the string directly from the response.

Enhancements to @Deprecated annotation

`Deprecated` annotation is one of the three original built-in annotations in Java. Over the years, users realised that it is quite ambiguous in nature. It didn't provide a lot of information previously, like why something was deprecated; could it be used or would it be removed in future versions of the library; if it was to be removed, then at which version would this occur, etc. So, now the deprecated annotation provides additional fields like `forRemoval` and `since` for more lucidity. There was another small enhancement whereby, if you were using a deprecated class and suppressed the warning, there would still be a warning left in the place it was used due to the import statement, and there was no way to annotate the import statement. This issue has also been fixed in the current release of Java 9.

Multi-release JAR files

Multiple, Java-release-specific versions of class/resource files can now coexist in the same JAR file. This is good news for library maintainers, who will not be limited to

using the minimum subset of the API provided by the least version of Java they want to support.

A multi-release JAR file is one whose `MANIFEST.MF` file includes the entry `Multi-Release: true` in its main section. Also, `META-INF` contains a versions subdirectory whose integer-named subdirectories — starting with 9 (for Java 9) — store version-specific class and resource files. JEP 238 offers the following (enhanced) example:

```
JAR content root
A.class
B.class
C.class
D.class
META-INF
    MANIFEST.MF
    versions
        9
            A.class
            B.class
```

If this JAR is read by JDK 9, it will use classes A and B for Java 9 inside the 9 directory, but if it is read by a pre-Java 9 JDK, it will read classes from the root.

Other updates in Java 9

Java 9 packs several other updates. For instance, `JavaDocs` now supports HTML 5 and the search feature, and new methods have been added to the `Stream` interface: `dropWhile`, `takeWhile`, `ofNullable`. The `iterate` method now also allows you to provide a predicate on when to stop iterating.

```
IntStream.iterate(1, i -> i <= 10, i -> i + 1).forEach(System.
out::println);
```

The `collections` framework has been updated to now allow a population of collections at the time of creation, as shown below:

```
List<Integer> integers = List.of(1, 2, 3);
```

There are several other new features in Java 9. To get a comprehensive list of all the new additions to Java 9, you can visit the release notes at <https://docs.oracle.com/javase/9/whatsnew/toc.htm>. 

By: Shiva Saxena

The author is a FOSS enthusiast. He currently works as a consultant, and is involved in developing enterprise application and Software-as-a-Service (SaaS) products. He has hands-on development experience with Android, Apache Camel, C#, .NET, Hadoop, HTML5, Java, OData, PHP, React, etc, and loves to explore new and bleeding-edge technologies. He can be reached at shivasaxena@outlook.com.